# EarthCARE Dense-Track Explainer

## What changed

The original 3D-cloud task assumed every target curtain already had the same shape. Your EarthCARE data does not: each chip carries a variable-length along-track curtain with shape (L, 206). That breaks default batching because PyTorch cannot stack different L values into one batch tensor.

## New idea

Instead of resizing or padding the along-track curtain itself, the new path predicts a vertical profile at every ABI pixel. Each sample is rasterized into fixed tensors: input (14, 128, 128), label (206, 128, 128), and valid (206, 128, 128). Once every sample has those fixed shapes, the standard DataLoader collate works again.

## Raw sample structure

Observed local sample format:

chip: (128, 128, 22) float32, HWC
data["Cloud_mask"]: (L, 206) bool
data["chip_row_idx"]: (L,) int32
data["chip_col_idx"]: (L,) int32
data["cloud_mask_cpr_atlid_msi_1km"]: (L, 206), NaNs mark invalid bins
data["height"]: (L, 206), 20000 m to -500 m in 100 m steps

Channel 22 is solar zenith angle and is used for day/night plotting only.

## Default model channels

The model input uses the requested 14 ABI channels in this exact order:

[2, 3, 1, 5, 6, 7, 4, 9, 10, 11, 12, 14, 15, 16]

Those are selected from the raw 22-channel chip before the existing ABI transform runs.

Files: pytorch_caney/datasets/abi_3dcloud_track_dataset.py and pytorch_caney/three_d_cloud_track_utils.py

# How One Sample Becomes Dense

## Step 1: build the input tensor

Start from chip = (128, 128, 22). Select the 14 requested ABI channels, then apply AbiToaTransform. The final model input for one sample is a tensor with shape (14, 128, 128).

## Step 2: allocate dense targets

Before rasterization, the dataset creates:
label = zeros((206, 128, 128), dtype=float32)
valid = zeros((206, 128, 128), dtype=bool)

So every height bin at every pixel starts as unsupervised.

## Step 3: map track profiles into chip pixels

Each EarthCARE profile i has:
row = chip_row_idx[i]
col = chip_col_idx[i]
curtain row = Cloud_mask[i, :] with shape (206,)

For valid bins only, the dataset writes that profile into label[:, row, col] and marks valid[:, row, col] = True.

## Step 4: drop or aggregate as needed

Profiles outside the chip are ignored. If multiple profiles land in the same ABI pixel, they are combined per height bin. Default aggregation is majority vote, with support for any and threshold modes too.

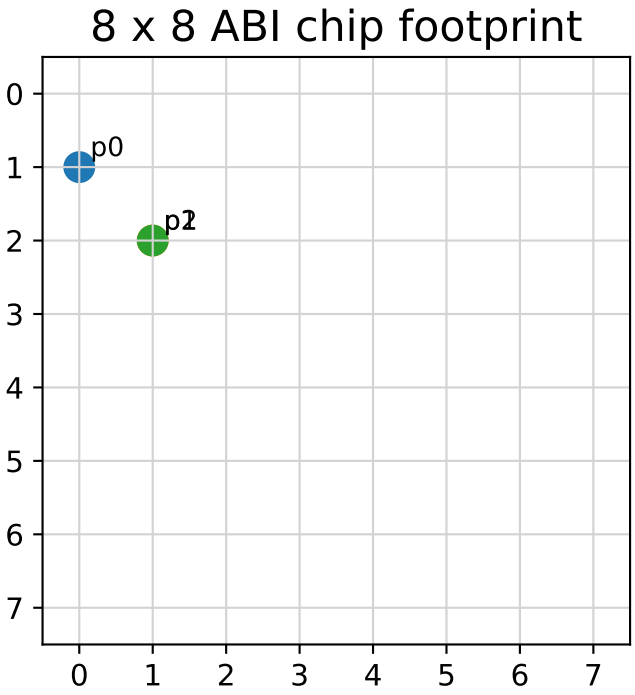
## Why this fixes batching

The variable dimension L disappears after rasterization. Every sample now has the same fixed dense shapes, so batch stacking succeeds.

The dense batch is used by the new datamodule abitoa3dcloud_track and pipeline 3dcloud_track.

# Toy Rasterization Example

Three variable-length track profiles are mapped into a fixed dense (Z, H, W) label/valid volume.




```
Input curtain profiles (L = 3, Z = 4):

p0 @ (1, 0): [1, 0, 1, 0]
p1 @ (2, 1): [0, 1, 1, 0]
p2 @ (2, 1): [1, 1, 0, 0]

Two profiles hit pixel (2, 1), so that pixel is
per height bin using the configured rule (defaul
```

```
Dense result at supervised pixels:

label[:, 1, 0] = [1, 0, 1, 0]
valid[:, 1, 0] = [1, 1, 1, 1]

label[:, 2, 1] = majority([0,1,1,0], [1,1,0,0]) = [0, 1, 0, 0]
valid[:, 2, 1] = [1, 1, 1, 1]

Everything off-track stays:
label[:, row, col] = 0
valid[:, row, col] = False

That is why the batch label tensor is mostly zeros, but only the
small set of valid bins contributes to the loss and metrics.
```

# How to Read the Batch Printout

## Example batch output

```
batch: list len=3
  [0]: tensor shape=(8, 14, 128, 128) dtype=torch.float32 ...
  [1]: tensor shape=(8, 206, 128, 128) dtype=torch.float32 ...
  [2]: tensor shape=(8, 206, 128, 128) dtype=torch.bool true_count=115726
Summary: inputs=(8, 14, 128, 128) labels=(8, 206, 128, 128) valid=(8, 206, 128, 128)
label_positive_count=4859 valid_count=115726
```

## Meaning

Batch size is 8. Each sample contributes a 14-channel ABI input chip plus dense label/valid tensors with 206 vertical bins over the 128 x 128 chip. The valid mask is sparse because EarthCARE only observes a narrow track through each chip.

## Why the label mean is tiny

The label tensor contains the full dense canvas, so most positions are off-track zeros. That is expected. Those off-track positions do not affect training because valid=False there.

## Counts to pay attention to

label_positive_count = number of cloud-positive bins in the batch.
valid_count = number of bins that actually contribute to the loss and metrics.

With 8 samples, the theoretical dense tensor size is 8 x 206 x 128 x 128 = 27,000,832 bins, but only a tiny fraction are supervised.

## Loss and metrics

The masked BCE loss multiplies the per-bin loss by valid and divides by valid.sum(). Metrics flatten only the valid positions and ignore the rest. So the model trains only where EarthCARE actually observed the atmosphere.